



Conceitos de Teste de Software

Além dos métodos utilizados para gestão de projetos de desenvolvimento de software, podemos utilizar testes para auxiliar a verificar que o sistema ou partes dele estão implementados de acordo com sua especificação. Na especificação de um sistema, temos a definição de blocos menores que o compõem: requisitos funcionais e não funcionais. Desta forma, um dos motivos de se testar um software é verificar que um software atende aos seus requisitos (SOMMERVILLE, 2019).

Usar testes é uma forma de atuar na busca de erros no processo de desenvolvimento, para minimizar a quantidade de defeitos que irão detrair a experiência dos usuários. Com a complexidade crescente dos sistemas, atuar de forma proativa para a identificação e correção de erros (bugs) pode contribuir de forma significativa para a qualidade do produto de software desenvolvido. Por isso, podemos considerar que outro motivo para testar um software é identificar e corrigir erros (SOMMERVILLE, 2019).

Durante o processo de desenvolvimento, podemos usar testes durante as atividades de verificação e validação. Estas atividades buscam garantir um nível de confiança no software. Este nível de confiança depende de sua finalidade, das expectativas de usuários e do ambiente de mercado em que o software está inserido (SOMMERVILLE, 2019). Por exemplo, um software para uma aplicação militar, médica ou nuclear pode ser muito mais crítica que um software para demonstração de uma tecnologia emergente em uma feira, e, portanto, o nível de confiança em aplicações críticas é maior. Usuários podem estar mais abertos a erros se são os

primeiros a testar uma nova tecnologia, considerando os early adopters da Difusão de Inovações (ROGERS et al., 2014).

Relembrando: enquanto a validação está preocupada em entender se estamos desenvolvendo o produto correto, o objetivo da verificação é garantir que estamos desenvolvendo o produto de software da forma correta (SOMMERVILLE, 2019). Dentre os testes de verificação, temos os testes de unidade, de integração e de sistema. Já dentre os testes de validação, temos os testes alfa e beta (GALLOTTI, 2015).

Podem ser denominados testes estáticos (ou análises estáticas) as inspeções de código realizadas no próprio código fonte do software, sua especificação de requisitos ou modelos de arquitetura. Dentre suas vantagens, temos que as inspeções podem descobrir muitos erros de uma só vez, sem o problema de que um erro pode encobrir outro (o que pode ocorrer em um teste dinâmico). Com a inspeção, é mais fácil entender se partes incompletas do sistema estão operando de forma correta, antes mesmo de sua implementação completa. Por fim, as inspeções também permitem verificar se o código está aderente a padrões e se a forma como foi construído favorece sua manutenção (SOMMERVILLE, 2019).

Contudo, os testes estáticos não podem substituir os testes dinâmicos, pois há defeitos causados por interações entre diferentes partes de um código, problemas com o desempenho de um sistema ou de temporização que só podem ser identificados com a execução de testes dinâmicos. Dentre os testes dinâmicos, temos os testes de desenvolvimento realizados pelos desenvolvedores do sistema, testes de lançamento realizados por equipes especializadas em testes para evitar o viés do autor, testar seu artefato e obter uma percepção mais imparcial sobre o sistema, e um teste de usuário, em que o teste é realizado no seu ambiente de implantação (SOMMERVILLE, 2019).

Após entender todos estes conceitos relacionados aos testes, podemos verificar que não é algo simples, trivial de se realizar. De nada servem testes ingênuos que não ajudam a investigar a existência de comportamentos anômalos no software. Quando realizamos um teste, temos condições prévias (estado do sistema antes do teste), a manipulação de entradas conforme algum procedimento de teste, a observação das saídas e a comparação deste comportamento observado com o comportamento esperado (descrito no formato de saídas esperadas). Durante todo este ciclo, é importante sempre lembrar qual o intuito do teste, em que fase do desenvolvimento e qual sua importância para sempre melhorar o procedimento do teste. Não basta realizar testes sem alguma reflexão associada, pois a execução de testes de forma indiscriminada pode levar ao consumo de recursos (tempo) sem resultados efetivos.

Por fim, é importante destacar que testar um software pode ajudar na identificação de erros, mas não garante que ele está isento de erros (SOMMERVILLE, 2019). Desta forma, realizar mais testes, mais diversos, pode contribuir para aumentar a cobertura de testes de um software, mas sabemos que esta cobertura nunca será de 100%. Caso contrário, seria possível construir um software e garantir que ele nunca apresentará algum bug, o que infelizmente não é possível.

Atividade Extra

Leia mais sobre a difusão de inovações (ROGERS et al., 2014). Reflita sobre as atitudes de early adopters e late majority perante erros de uma nova

tecnologia, e como isso faz com que mais ou menos testes sejam necessários, a depender do nível de adoção de determinada tecnologia.

Referências Bibliográficas

GALLOTTI, G. M. A. (Org.). **Qualidade de software**. Pearson: 2015.

ROGERS, Everett M.; SINGHAL, Arvind; QUINLAN, Margaret M. **Diffusion of innovations**. In: An integrated approach to communication theory and research. Routledge, 2014. p. 432-448.

SOMMERVILLE, I. **Engenharia de Software**, 10ª edição. Addison Wesley, 2019.

Ir para exercício